SOFTWARE ENGINEERING · CHAPTER 11

Component Diagrams

Component diagrams show the organization and dependencies of deployable or replaceable software units. They provide an architectural view of how components collaborate through interfaces.

UML 2.5.1 | JDK 25 LTS · JPMS | Maven 3.9 · Spring Boot 4.1 | PlantUML · C4

AGENDA

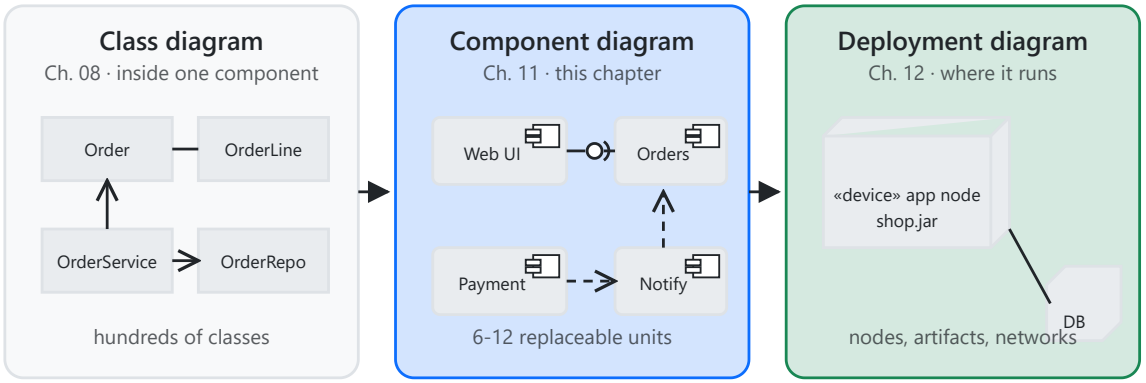
What we will cover

1. Components, ports, and provided interfaces
2. Required interfaces and connectors
3. Dependencies and component boundaries
4. Layered and modular architectures
5. Services, libraries, databases, and external systems
6. Mapping components to source modules
7. Dependency direction and coupling
8. Deployment and integration concerns
9. Architectural documentation and evolution
10. Validating component responsibilities
11. Chapter Quiz
12. Lab-11

Click any item to jump directly to that topic.

From classes to deployable units

- Chapters 08-10 modelled **classes** and their **interactions**. A real system has hundreds of classes: nobody can reason about them one by one.
- A **component** is a modular, *replaceable* unit that hides its internals and is known only through its **interfaces** (UML 2.5.1, §11.6).
- Component diagrams answer: *What are the parts? What does each offer and need? Who depends on whom?*
- They are the bridge between the code model (packages, Maven modules) and the physical model (Chapter 12: deployment).
- Running example: the **online shop**, first as a **modular monolith**, later as **microservices**.



zoom out: same system, coarser units, fewer boxes

a component groups classes that change together and are deployed together

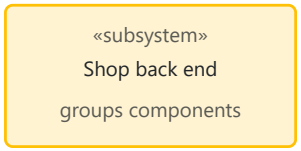
UML 2.5.1 component notation at a glance

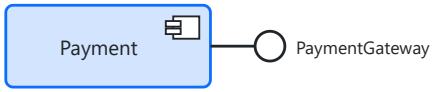


component: keyword and/or icon



port: interaction point on the boundary

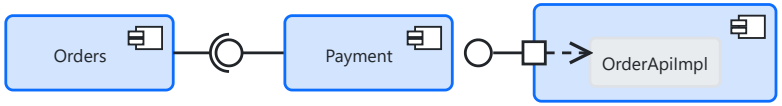




provided interface: lollipop ○— (offers)

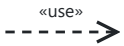


required interface: socket —((needs)

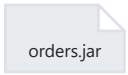


assembly connector: ball in socket

delegation: port → internal part



dependency (dashed, open head)

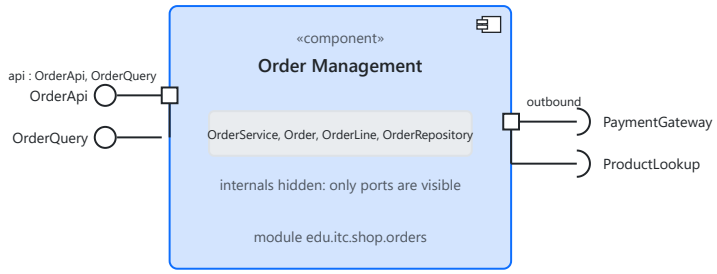


«artifact»
«manifest»

- A **component** is a rectangle with the `«component»` keyword and/or the two-small-rectangles **icon**.
- A **port** (small square on the border) is a named interaction point; interfaces attach to ports or to the component edge.
- **Provided** interface: lollipop `○–`. **Required** interface: socket `–(`.
- **Assembly connector**: a socket plugged onto a lollipop wires a requirement to a provision.
- **Delegation connector**: forwards a port to an internal part that implements it.
- **Artifacts** (`orders.jar`) *manifest* components; `«subsystem»` groups them.

```
@startuml
component "Order Management" as OM
interface "OrderApi" as IOrder
interface "PaymentGateway" as IPay
IOrder - OM
OM ..> IPay : uses
@enduml
```

One component, two ports: what it offers, what it needs



provided on the left (lollipops), required on the right (sockets) is a convention, not a rule

```java
// Provided interface: what Order Management OFFERS (port "api")
package edu.itc.shop.orders.api;

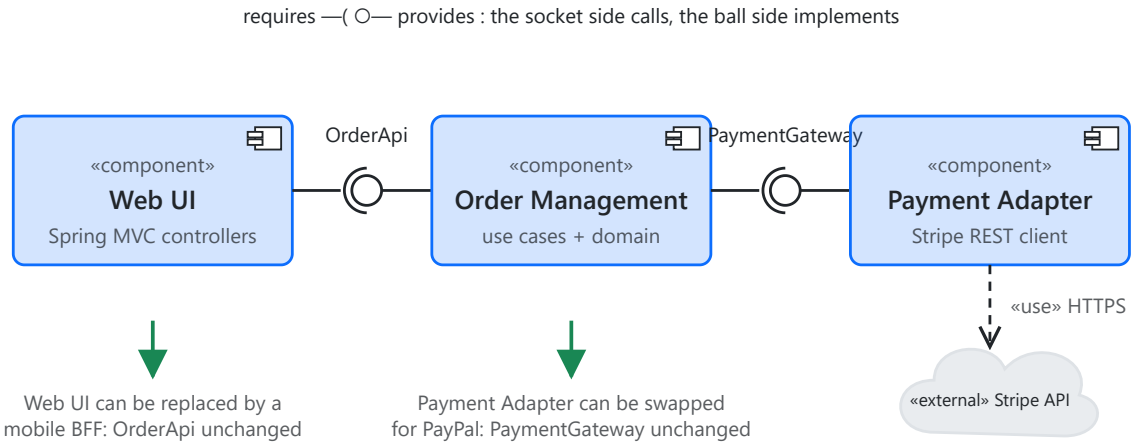
public interface OrderApi {
    OrderId place(PlaceOrderCommand cmd);
    void cancel(OrderId id);
}
public interface OrderQuery {
    Optional<OrderView> byId(OrderId id);
    Page<OrderView> forCustomer(CustomerId c, Pageable p);
}

// Required interfaces: what it NEEDS (port "outbound"),
// declared BY the orders module, implemented by somebody else
package edu.itc.shop.orders.spi;

public interface PaymentGateway {
    PaymentResult charge(OrderId id, Money amount, PaymentToken token);
}
public interface ProductLookup {
    Optional<ProductSnapshot> find(ProductId id);
}
```

The interface *type* is the same Java `interface` either way; provided vs required is a matter of which component **implements** it and which one **calls** it.

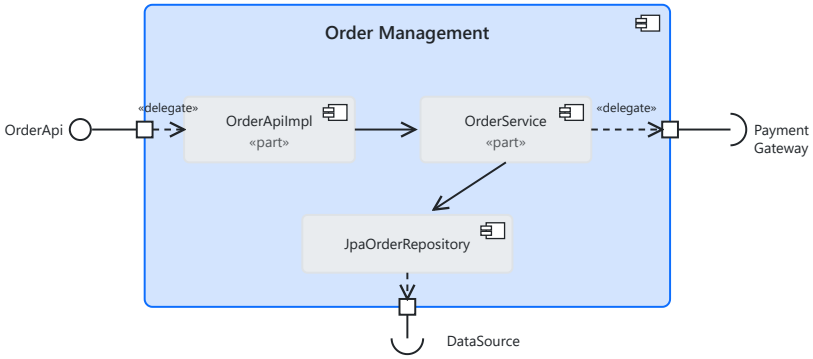
Assembly connectors: plugging requirements into provisions



- An **assembly connector** joins a required interface to a compatible provided one; the notation is the ball sitting in the socket.
- Read it as a *contract*: the caller knows only the interface, never the implementing class.
- A required interface with **no** connector means the system is incomplete (or the provider is external).
- One provided interface may serve **many** sockets; one socket is wired to **exactly one** ball.

```
@startuml
component "Web UI" as UI
component "Order Management" as OM
component "Payment Adapter" as PA
cloud "Stripe API" as Stripe
interface OrderApi
interface PaymentGateway
UI -( OrderApi
OrderApi - OM
OM -( PaymentGateway
PaymentGateway - PA
PA ..> Stripe : HTTPS
@enduml
```

Delegation connectors and their Java twin: `module-info.java`



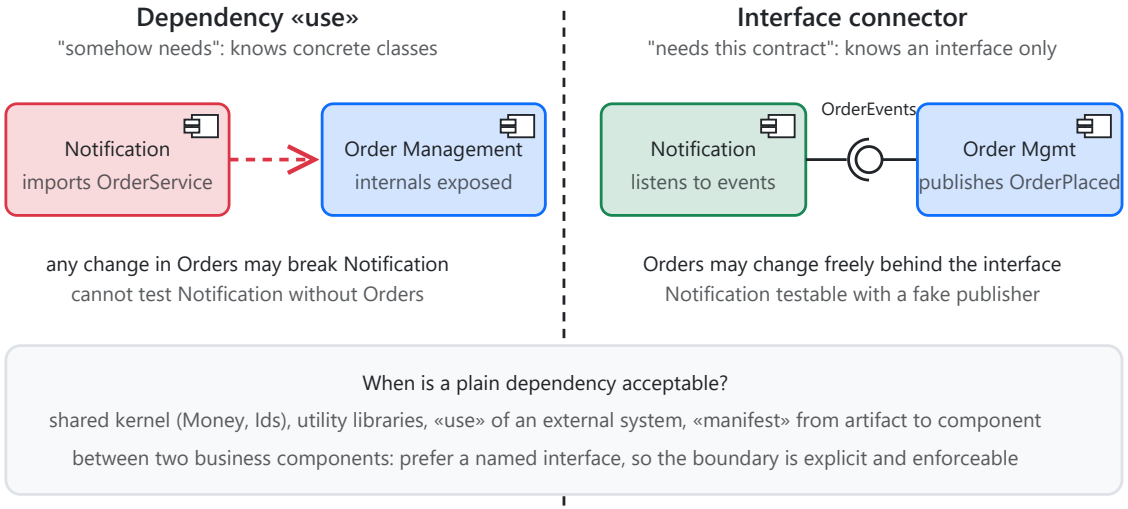

```
// library-orders/src/main/java/module-info.java  (JDK 25, JPMS)
module edu.itc.shop.orders {
    // provided interfaces: the ONLY packages other modules may see
    exports edu.itc.shop.orders.api;
    // required interfaces live in this module too (spi = service
    // provider interface); adapters in other modules implement them
    exports edu.itc.shop.orders.spi;

    // required dependencies (the sockets, at module level)
    requires edu.itc.shop.catalog;
    requires transitive edu.itc.shop.shared;
    requires spring.context;
    requires jakarta.persistence;

    // let Spring/Hibernate reflect on internal classes without
    // making them part of the public contract
    opens edu.itc.shop.orders.internal to spring.core, spring.beans;
    opens edu.itc.shop.orders.internal.persistence to org.hibernate.orm.core;
}
```

UML	JPMS
provided interface / port	`exports` package with the interfaces
required interface	`requires` module (or an exported `spi` package)
delegation to internal part	non-exported package, wired by Spring

Dependency arrows versus interface connectors



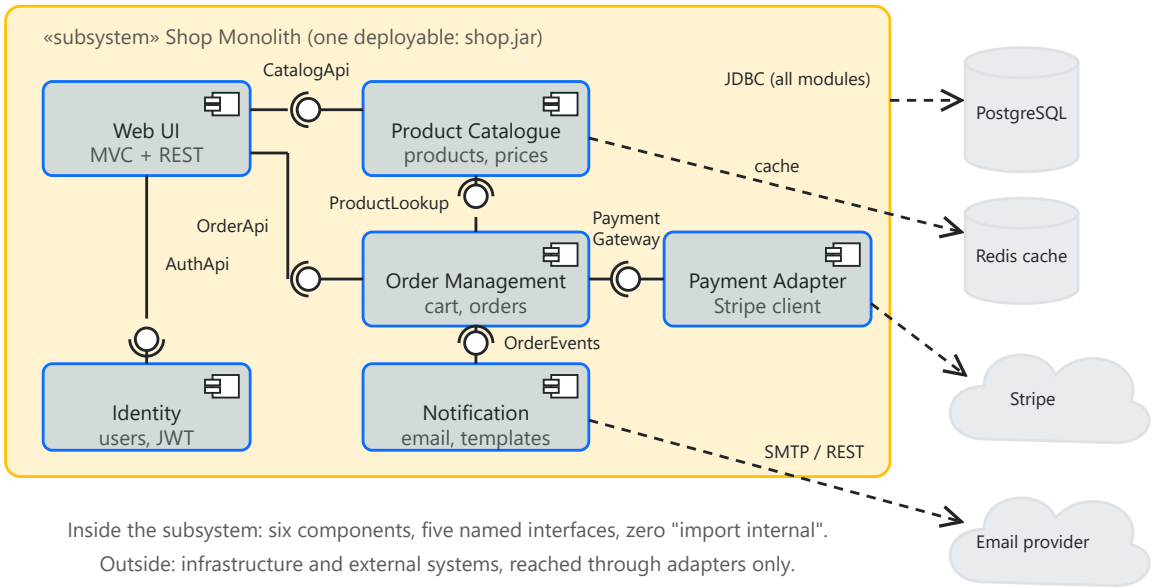
Notation	Meaning	Java
`A ..> B` dashed	A uses something in B (unspecified)	`import b.*`
`A -( I`, `I - B`	A needs contract I, B fulfils it	`interface I`, `B implements I`
`A ..> B : «use»`	call into an external system	HTTP/SDK client
`A ..> B : «manifest»`	artifact A materialises component B	`orders.jar`

```
// The event is the contract; Notification never
// imports edu.itc.shop.orders.internal
package edu.itc.shop.orders.api;

public record OrderPlaced(OrderId id,
                          CustomerId customer,
                          Money total,
                          Instant at) {}
```

⚠ A dashed arrow into a component's *internal* package is a boundary violation, whatever the diagram says.

Drawing the boundary: the online shop as a modular monolith



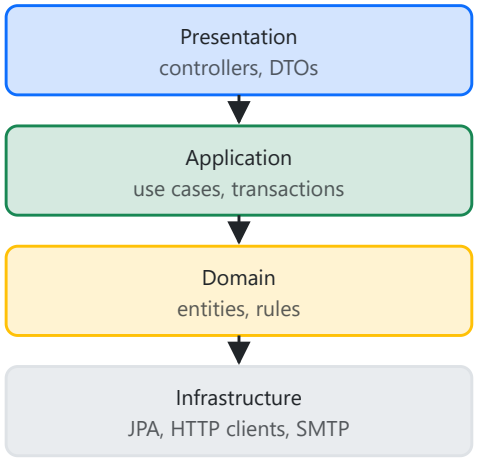
Inside the subsystem: six components, five named interfaces, zero "import internal".
Outside: infrastructure and external systems, reached through adapters only.

- **Boundary rule**: a component owns its data model and exposes it only through interfaces and events.
- Group classes that **change for the same reason** (cohesion); split where change rates differ.
- Infrastructure (PostgreSQL, Redis) and external systems (Stripe, email) sit **outside** the subsystem, connected by «use» dependencies from adapters.
- The same diagram stays valid when a component is later extracted into a service: only the connector's transport changes.

```
package "Shop Monolith" <<subsystem>> {
  component "Web UI" as UI
  component "Product Catalogue" as Cat
  component "Order Management" as OM
  component "Payment Adapter" as Pay
  component "Notification" as Not
  component Identity
}
database PostgreSQL
database "Redis cache" as Redis
cloud Stripe
UI -( OrderApi
OrderApi - OM
OM ..> PostgreSQL
```

Two ways to organise the inside of a component

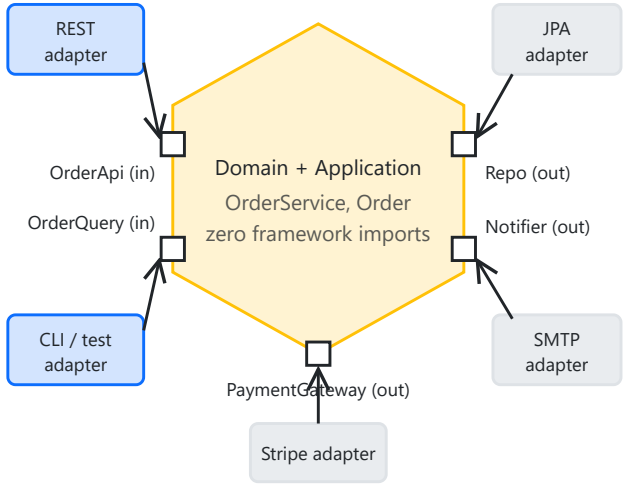
Layered (Ch. 02)



strict: each layer calls only the one below
risk: domain ends up depending on JPA

Hexagonal (ports and adapters)

all arrows point INTO the hexagon



- **Layered**: horizontal slices; dependencies point down. Simple, but the domain layer tends to inherit persistence details.
- **Hexagonal** (Cockburn): the domain sits in the middle and declares **ports** (interfaces). **Adapters** implement them at the edge. Every dependency points inwards.
- **Driving** adapters call inbound ports (REST, CLI, tests). **Driven** adapters implement outbound ports (JPA, SMTP, Stripe).
- Both are internal structure; a component diagram shows the ports either way. **Modular** means the shop consists of several such components with explicit interfaces.

	Layered	Hexagonal
Domain depends on	infrastructure (down)	nothing
Swap database	touch domain	new adapter
Unit-test domain	needs mocks of JPA	fake ports
Learning curve	low	medium

A port, its adapter, and the Spring wiring that plugs them together

```java
// PORT (required interface), owned by the orders module.
// Speaks the domain language: Money, OrderId, no Stripe types.
package edu.itc.shop.orders.spi;

public interface PaymentGateway {
    PaymentResult charge(OrderId order, Money amount,
                         PaymentToken token);

    sealed interface PaymentResult
            permits Approved, Declined {}
    record Approved(String reference) implements PaymentResult {}
    record Declined(String reason)    implements PaymentResult {}
}

// ADAPTER (driven side), in the payment module.
// Translates the port into Stripe's REST API.
package edu.itc.shop.payment.stripe;

public class StripePaymentAdapter implements PaymentGateway {
    private final RestClient stripe;
    private final String secretKey;

    public StripePaymentAdapter(RestClient stripe, String key) {
        this.stripe = stripe; this.secretKey = key;
    }

    @Override
    public PaymentResult charge(OrderId order, Money amount,
                                PaymentToken token) {
        var body = Map.of("amount", amount.minorUnits(),
                          "currency", amount.currency().code(),
                          "source", token.value(),
                          "metadata[orderId]", order.value());
        var res = stripe.post().uri("/v1/charges")
                .header("Authorization", "Bearer " + secretKey)
                .body(body).retrieve().body(StripeCharge.class);
        return res.paid()
            ? new Approved(res.id())
            : new Declined(res.failureMessage());
    }
}
```

```java
// WIRING: the application module decides which adapter
// fills each socket. The domain never sees this class.
package edu.itc.shop.app.config;

@Configuration(proxyBeanMethods = false)
public class PaymentConfig {

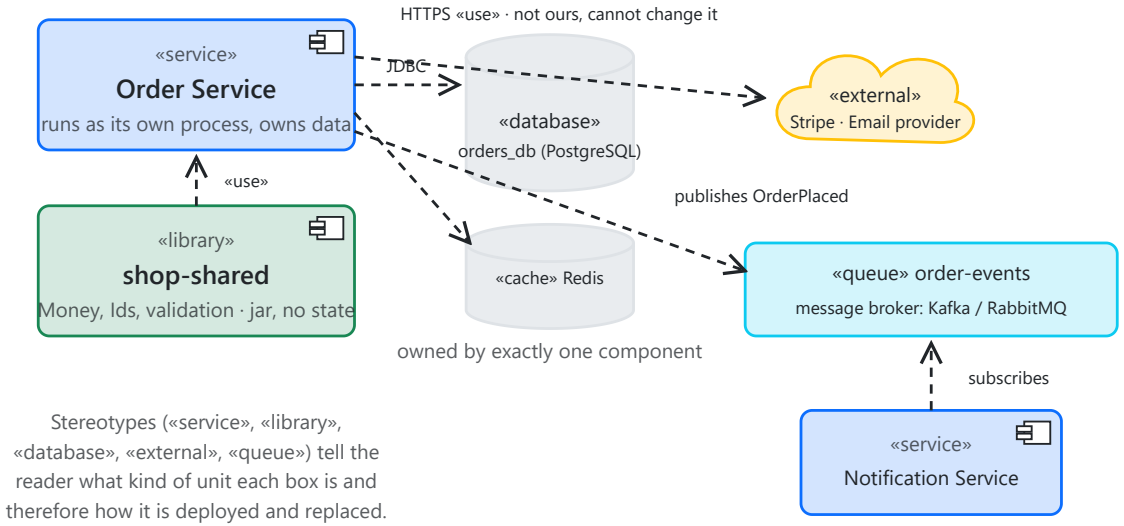
    @Bean
    @Profile("!test")
    PaymentGateway stripePaymentGateway(
            RestClient.Builder builder,
            @Value("${shop.stripe.key}") String key) {
        var client = builder.baseUrl("https://api.stripe.com")
                            .build();
        return new StripePaymentAdapter(client, key);
    }

    @Bean
    @Profile("test")
    PaymentGateway fakePaymentGateway() {
        // approves everything: tests the orders component alone
        return (order, amount, token) ->
            new PaymentGateway.Approved("test-" + order.value());
    }
}
```

☑ The `@Configuration` class is the **assembly connector** in code: it plugs the `StripePaymentAdapter` ball into the `PaymentGateway` socket. Swapping to PayPal changes this class and adds one adapter, nothing else.

⚠ If `OrderService` imports `StripeCharge`, the port has leaked: the diagram shows a socket, but the code has a hard dependency.

Not every box is the same kind of component

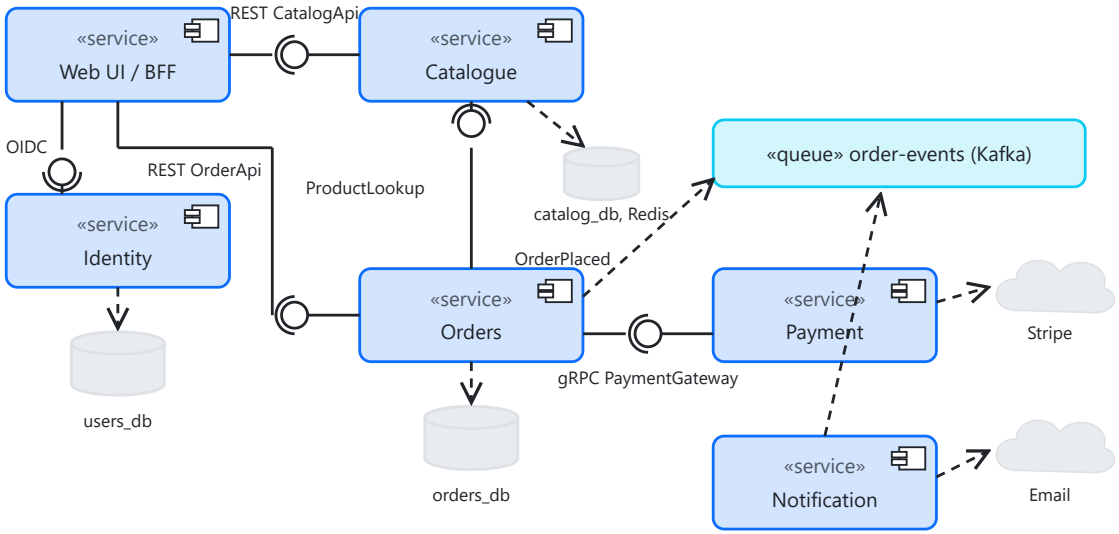


Stereotypes («service», «library», «database», «external», «queue») tell the reader what kind of unit each box is and therefore how it is deployed and replaced.

Kind	Unit of deployment	Replaced by	PlantUML
«service»	process / container	redeploy, API kept	`component`
«library»	jar on classpath	version bump	`component` / `artifact`
«database»	schema, owned by one component	migration	`database`
«external»	someone else's system	new adapter	`cloud`
«queue»	topic / exchange	contract = event schema	`queue`
node	machine, VM, container	Chapter 12	`node`

```
@startuml
component "Order Service" <<service>> as OS
component "shop-shared" <<library>> as Lib
database "orders_db" as DB
cloud "Stripe" as Stripe
queue "order-events" as Q
node "k8s pod" { component "Notification" as N }
OS ..> Lib
OS ..> DB : JDBC
OS ..> Stripe : HTTPS
OS ..> Q : publishes
Q ..> N : subscribes
@enduml
```

The same components as microservices: what changes, what does not

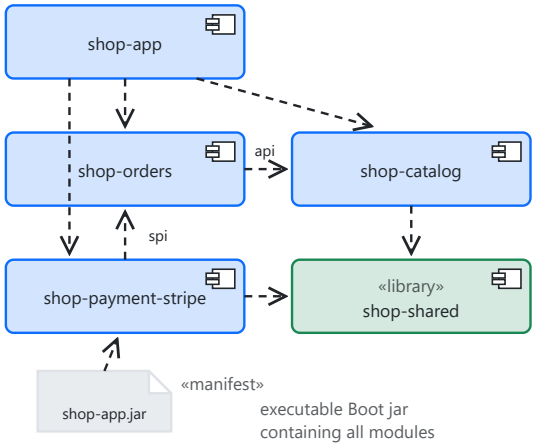


same six components, same interfaces; connectors now cross process boundaries and each service owns its database

	Modular monolith	Microservices
Component	Maven module / JPMS module	deployable service
Interface	Java interface	REST / gRPC / event schema
Connector	method call, Spring bean	HTTP client, broker
Database	one instance, schema per module	one per service
Transaction	local ACID	sagas, eventual consistency
Failure	exception	timeout, retry, circuit breaker
Refactor a boundary	IDE move	API versioning, migration

Start with the monolith and **get the component diagram right**. Extracting a service later is cheap only if the boundary already exists in code; that is what the rest of this chapter is about.

One component = one Maven module = one JPMS module




```
shop/ (reactor pom)
├── pom.xml <modules>
├── shop-shared/
│   └── module-info.java
├── shop-catalog/
│   ├── ..catalog.api
│   └── ..catalog.internal
├── shop-orders/
│   ├── ..orders.api (exports)
│   ├── ..orders.spi (exports)
│   └── ..orders.internal
├── shop-payment-stripe/
│   └── implements orders.spi
└── shop-app/ @SpringBootApplication
└── config/ wiring, profiles
```

```xml
<!-- shop/pom.xml : the reactor (aggregator + parent) -->
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>4.1.1</version>
  </parent>
  <groupId>edu.itc.shop</groupId>
  <artifactId>shop</artifactId>
  <version>1.0.0</version>
  <packaging>pom</packaging>
  <properties><java.version>25</java.version></properties>

  <!-- one Maven module per component, build order follows deps -->
  <modules>
    <module>shop-shared</module>
    <module>shop-catalog</module>
    <module>shop-orders</module>
    <module>shop-payment-stripe</module>
    <module>shop-app</module>
  </modules>
</project>

<!-- shop-payment-stripe/pom.xml : the socket it fills -->
<dependency>
  <groupId>edu.itc.shop</groupId>
  <artifactId>shop-orders</artifactId>
  <version>${project.version}</version>
</dependency>
```

Maven `<dependency>` = the dashed arrow; JPMS `exports` = the lollipop. Together they make the diagram **compiler-enforced**.

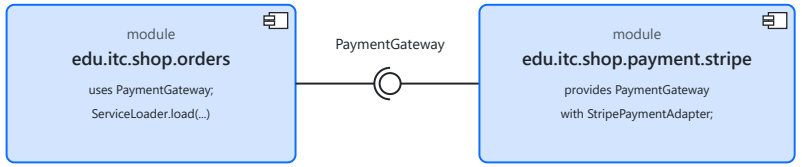
Provided and required at the JDK level: `provides … with` and `ServiceLoader`

```
// shop-orders/module-info.java : declares the socket
module edu.itc.shop.orders {
    exports edu.itc.shop.orders.spi;
    uses edu.itc.shop.orders.spi.PaymentGateway;   // "I require one"
}

// shop-payment-stripe/module-info.java : fills the socket
module edu.itc.shop.payment.stripe {
    requires edu.itc.shop.orders;                 // sees the spi package
    requires spring.web;
    provides edu.itc.shop.orders.spi.PaymentGateway
        with edu.itc.shop.payment.stripe.StripePaymentAdapter;
}

// Inside the orders module: resolve the provider at runtime,
// without importing a single Stripe class (no spring.factories,
// no reflection on class names, just the module graph).
public final class Gateways {
    public static PaymentGateway payment() {
        return ServiceLoader.load(PaymentGateway.class)
                .findFirst()
                .orElseThrow(() -> new IllegalStateException(
                    "no PaymentGateway provider on the module path"));
    }
}

// Provider needs a public no-arg constructor or a
// public static PaymentGateway provider() method:
public class StripePaymentAdapter implements PaymentGateway {
    public static PaymentGateway provider() {
        return new StripePaymentAdapter(RestClient.create(),
                                        System.getenv("STRIPE_KEY"));
    }
    /* ... */
}
```



the module system, not the code, wires ball to socket

swap the provider module on the module path, orders stays untouched

UML	module-info
required interface	`uses I;`
provided interface	`provides I with Impl;`
assembly connector	`ServiceLoader` or Spring `@Bean`

Spring Modulith: components as packages, verified by a test

```
// One top-level package per component under the Boot application
// package. Sub-packages are internal unless exposed.
edu.itc.shop                    ShopApplication
edu.itc.shop.orders             OrderApi, OrderPlaced   (module API)
edu.itc.shop.orders.internal    OrderService, JpaOrderRepository
edu.itc.shop.catalog            ProductLookup           (module API)
edu.itc.shop.payment            PaymentGateway impl
edu.itc.shop.notification       listens to OrderPlaced
```

```
// edu/itc/shop/orders/package-info.java
@org.springframework.modulith.ApplicationModule(
    displayName = "Order Management",
    allowedDependencies = { "catalog", "shared" })
package edu.itc.shop.orders;
```

```
// Publishing the event = the provided OrderEvents interface
@Service
class OrderService implements OrderApi {
    private final ApplicationEventPublisher events;
    /* constructor ... */
    @Transactional
    public OrderId place(PlaceOrderCommand cmd) {
        var order = Order.from(cmd);
        repo.save(order);
        events.publishEvent(new OrderPlaced(order.id(), cmd.customer(),
                                            order.total(), Instant.now()));
        return order.id();
    }
}
```

```
// The test that FAILS when a boundary is crossed
class ModularityTests {
    ApplicationModules modules = ApplicationModules.of(ShopApplication.class);

    @Test void verifiesModuleStructure() { modules.verify(); }

    @Test void writesDocumentation() {
        new Documenter(modules).writeDocumentation();   // C4 + PlantUML
    }
}
```

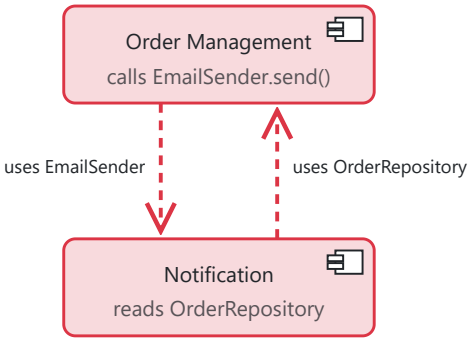
- **Convention**: each direct sub-package of the application package is an *application module*; its root package is the API, its sub-packages are internal.
- `allowedDependencies` makes the required interfaces explicit; any other cross-module import fails `verify()`.
- **Events** (`@ApplicationModuleListener`) decouple modules without a required interface; the event record is the contract.
- `Documenter` generates a component diagram (PlantUML, C4 style) from the code, so the diagram cannot drift.

```
<dependency>
  <groupId>org.springframework.modulith</groupId>
  <artifactId>spring-modulith-starter-core</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.modulith</groupId>
  <artifactId>spring-modulith-starter-test</artifactId>
  <scope>test</scope>
</dependency>
```

⚠ `verify()` also fails on **cycles** between modules. That is the topic of the next slide.

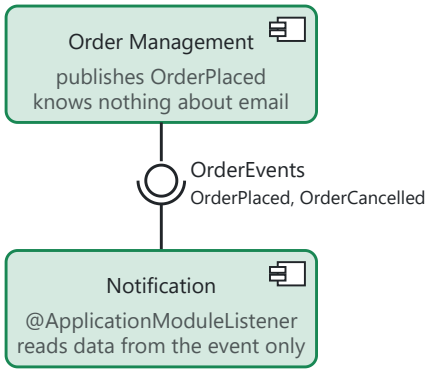
Dependencies must form a DAG: spotting a cycle and breaking it

Before: a cycle



A → B → A: neither can be built, tested or deployed alone. Maven refuses the reactor, JPMS refuses the module graph.

After: dependency inversion



Arrows point one way, towards the stable module. Orders is built, tested and deployed first; Notification can be removed without touching it.

Alternative: extract the shared part into a third module.

- **Acyclic Dependencies Principle**: the component graph must have no cycles.
- **Stable Dependencies Principle**: depend in the direction of stability; a module many others depend on must change rarely.
- **Stable Abstractions Principle**: the stable modules should consist of interfaces and value types (`api`, `spi`, `shared`).
- Breaking a cycle: **invert** with an interface or event owned by the callee, or **extract** the shared part.

Metric (Martin)	Meaning
Ca, Ce	afferent / efferent couplings (in / out)
I = Ce / (Ca + Ce)	instability, 0 stable … 1 unstable
A = abstract / total	abstractness
D = |A + I − 1|	distance from the main sequence; aim for 0

Turning the arrows into failing tests with ArchUnit

```java
// shop-app/src/test/java/edu/itc/shop/ArchitectureTests.java
@AnalyzeClasses(packages = "edu.itc.shop",
                importOptions = ImportOption.DoNotIncludeTests.class)
class ArchitectureTests {

    // 1. the domain never depends on the web layer
    @ArchTest
    static final ArchRule domain_is_web_free =
        noClasses().that().resideInAPackage("..domain..")
            .should().dependOnClassesThat()
            .resideInAnyPackage("..web..", "org.springframework.web..");

    // 2. only adapters may touch JPA
    @ArchTest
    static final ArchRule jpa_stays_in_adapters =
        classes().that().dependOnClassesThat()
            .resideInAPackage("jakarta.persistence..")
            .should().resideInAPackage("..internal.persistence..");

    // 3. components talk through api/spi packages only
    @ArchTest
    static final ArchRule internals_are_private =
        noClasses().that().resideOutsideOfPackage("..orders..")
            .should().dependOnClassesThat()
            .resideInAPackage("..orders.internal..");

    // 4. the component graph is acyclic
    @ArchTest
    static final ArchRule no_cycles =
        slices().matching("edu.itc.shop.(*)..")
            .should().beFreeOfCycles();

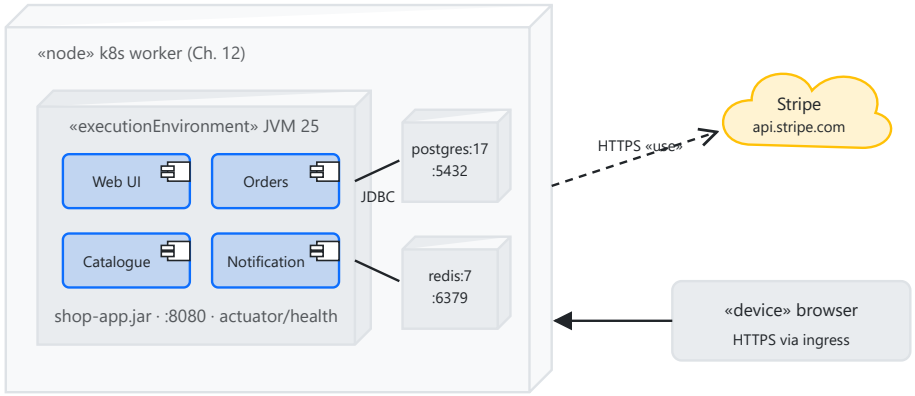
    // 5. layers inside a component (hexagonal flavour)
    @ArchTest
    static final ArchRule hexagon =
        onionArchitecture()
            .domainModels("..domain.model..")
            .domainServices("..domain.service..")
            .applicationServices("..application..")
            .adapter("web", "..adapter.web..")
            .adapter("persistence", "..adapter.persistence..")
            .adapter("payment", "..adapter.payment..");
}
```

```xml
<dependency>
  <groupId>com.tngtech.archunit</groupId>
  <artifactId>archunit-junit5</artifactId>
  <version>1.4.1</version>
  <scope>test</scope>
</dependency>
```

```
[ERROR] ArchitectureTests.internals_are_private
Architecture Violation [Priority: MEDIUM] -
Rule 'no classes that reside outside of package
'..orders..' should depend on classes that reside
in a package '..orders.internal..'' was violated
(1 times):
Class <...notification.OrderMailer> calls method
<...orders.internal.OrderService.find(OrderId)>
in (OrderMailer.java:31)
```

Coupling	Looks like	Prefer
Content	reads another module's entity / table	API call, event
Common	shared mutable static, shared schema	owned data
Control	flags that steer the callee's logic	separate operations
Data / message	DTOs, records, events	this is the goal

Where the components run, and testing the seams between them



deployment view: which artifact runs in which node; component view: what talks to what. Keep them consistent. every dashed line that leaves the JVM is an integration seam worth a test

```java
// Integration boundary test: real PostgreSQL in a container,
// Stripe replaced by the test-profile fake (slide 11).
@SpringBootTest
@Testcontainers
@ActiveProfiles("test")
class OrderPersistenceBoundaryIT {

    @Container
    @ServiceConnection
    static PostgreSQLContainer<?> db =
        new PostgreSQLContainer<>("postgres:17-alpine");

    @Autowired OrderApi orders;
    @Autowired OrderQuery query;
    @MockitoSpyBean PaymentGateway payment;

    @Test
    void placedOrderCrossesTheJpaAdapterAndBack() {
        var id = orders.place(new PlaceOrderCommand(
            new CustomerId(42), List.of(new Line(new ProductId(7), 2)),
            new PaymentToken("tok_test")));

        assertThat(query.byId(id)).isPresent()
            .get().extracting(OrderView::status)
            .isEqualTo(OrderStatus.PAID);
        verify(payment).charge(eq(id), any(), any());
    }
}
```

- **Integration concern**: each connector that crosses a process boundary needs a protocol, a timeout, a retry policy and a test.
- `@ServiceConnection` (Boot 4) wires the container's JDBC URL automatically; no `@DynamicPropertySource` needed.
- Test the *boundary*, not the internals: call the provided interface, assert through the provided query, verify the required interface was used.

Does the built artifact match the diagram? `jdeps` and `mvn dependency:analyze`

```
# 1. Module-level graph of the built jars (JDK 25)
$ jdeps --module-path shop-app/target/modules \
        --multi-release 25 -summary \
        shop-orders/target/shop-orders-1.0.0.jar
edu.itc.shop.orders -> edu.itc.shop.catalog
edu.itc.shop.orders -> edu.itc.shop.shared
edu.itc.shop.orders -> jakarta.persistence
edu.itc.shop.orders -> spring.context
edu.itc.shop.orders -> java.base

# 2. Package-level: who uses orders.internal from outside?
$ jdeps -verbose:package -e 'edu.itc.shop.orders.internal.*' \
        shop-notification/target/shop-notification-1.0.0.jar
   edu.itc.shop.notification -> edu.itc.shop.orders.internal
# (any line here is a boundary violation)

# 3. Generate a DOT graph and render it
$ jdeps --dot-output target/jdeps -summary shop-*/target/*.jar
$ dot -Tsvg target/jdeps/summary.dot -o target/components.svg

# 4. Do the poms match what the code really uses?
$ mvn -q dependency:analyze -DfailOnWarning=true
[WARNING] Used undeclared dependencies found:
[WARNING]    com.fasterxml.jackson.core:jackson-databind:jar
[WARNING] Unused declared dependencies found:
[WARNING]    edu.itc.shop:shop-payment-stripe:jar:1.0.0
```

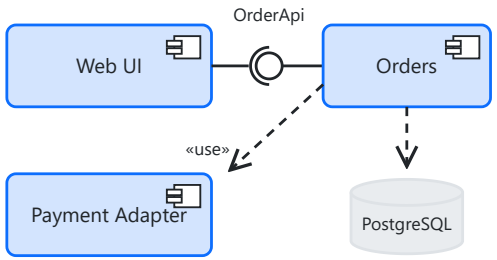
Finding	What it means for the diagram	Fix
`jdeps` edge not in diagram	hidden dependency	add it, or remove the import
edge into `..internal`	boundary breached	go through api/spi
Used undeclared	transitive luck; breaks when a neighbour changes	declare it
Unused declared	the socket in the pom is wired to nothing	remove, or move to `runtime` scope
cycle in `summary.dot`	ADP violated	invert (slide 17)

```
<!-- fail the build on drift: shop/pom.xml -->
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-dependency-plugin</artifactId>
  <executions>
    <execution>
      <id>analyze</id>
      <goals><goal>analyze-only</goal></goals>
      <configuration>
        <failOnWarning>true</failOnWarning>
        <ignoreNonCompile>true</ignoreNonCompile>
      </configuration>
    </execution>
  </executions>
</plugin>
```

Run these in CI. A component diagram that is checked by `verify()`, ArchUnit and `jdeps` is documentation that cannot lie.

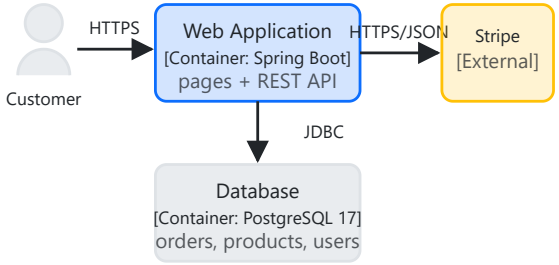
UML component diagram and C4 container diagram: same system, two audiences

UML 2.5 component (precise)



interfaces, ports, connectors
for designers and developers
shows WHAT is provided/required

C4 level 2: container (readable)



people, containers, technology labels
for stakeholders and new team members
shows WHERE the code runs and HOW it talks

C4 level 3 (component) is the UML component diagram of ONE container: Web UI, Orders, Catalogue inside "Web Application"
C4 = Context, Container, Component, Code

```
@startuml
!include <C4/C4_Container>
LAYOUT_WITH_LEGEND()

Person(customer, "Customer", "Buys products")
System_Boundary(shop, "Online Shop") {
  Container(web, "Web Application",
    "Spring Boot 4.1, Java 25",
    "Pages + REST API; modules: orders,
     catalog, payment, notification")
  ContainerDb(db, "Database", "PostgreSQL 17",
    "orders, products, users")
  ContainerQueue(bus, "order-events", "Kafka",
    "OrderPlaced, OrderCancelled")
}
System_Ext(stripe, "Stripe", "Card payments")
System_Ext(mail, "Email provider", "SMTP")

Rel(customer, web, "Uses", "HTTPS")
Rel(web, db, "Reads/writes", "JDBC")
Rel(web, bus, "Publishes", "Kafka")
Rel(web, stripe, "Charges", "HTTPS/JSON")
Rel(web, mail, "Sends", "SMTP")
@enduml
```

Use C4 for the first two levels of any architecture document and UML component diagrams when you need **interfaces and connectors** to be exact.

Recording decisions: ADRs, and a home for them in arc42

```
# ADR-007: Notification consumes OrderPlaced events

* Status: Accepted (2026-09-18)   supersedes ADR-003
* Deciders: shop team, architecture guild

## Context
Order Management called `EmailSender` directly and Notification
read `OrderRepository`, creating a cycle (ArchUnit `no_cycles`
fails). Notification must also be extractable as a service.

## Decision
Order Management publishes `OrderPlaced` / `OrderCancelled`
through Spring Modulith events. Notification listens with
`@ApplicationModuleListener` and reads only event data.
The event record is part of the `orders.api` package.

## Consequences
+ Orders no longer depends on Notification; cycle removed.
+ Externalised events (Kafka) later without touching Orders.
- Emails are sent asynchronously; failures need the event
  publication registry and a retry job.
- Event schema becomes a public contract: version it.

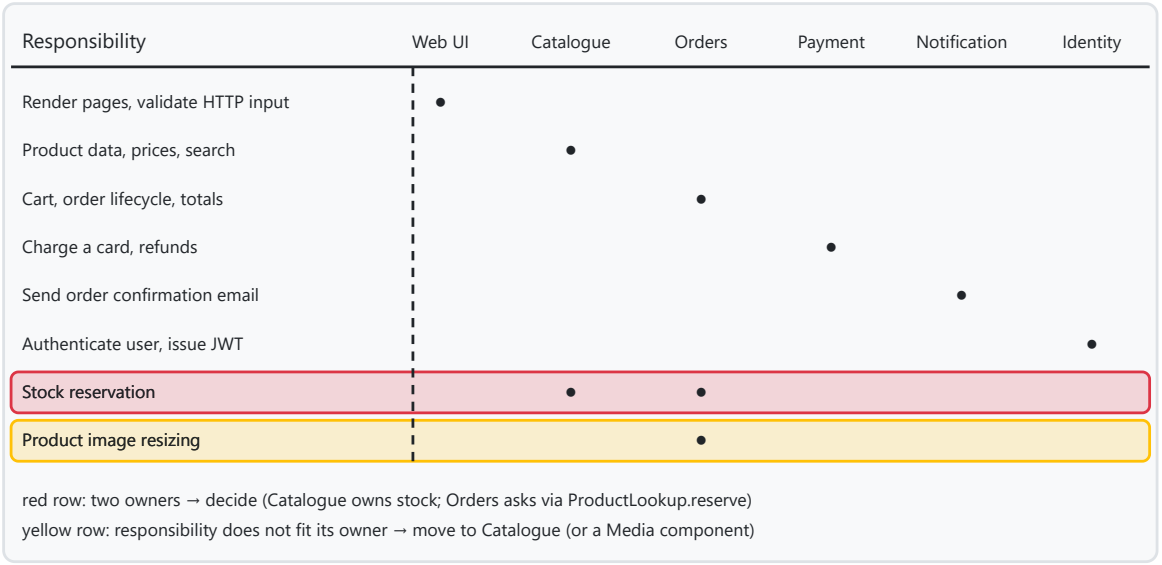
## Alternatives considered
1. Shared `OrderSummary` module: fixes the cycle but keeps
   synchronous coupling.
2. Notification calls `OrderQuery`: still a dependency on Orders.
```

```
docs/                          arc42 skeleton (12 sections)
├── 01-introduction-goals.md     top 3 quality goals, stakeholders
├── 02-constraints.md            JDK 25, Boot 4.1, PostgreSQL
├── 03-context-scope.md          C4 level 1 (system context)
├── 04-solution-strategy.md      modular monolith, hexagonal
├── 05-building-blocks.md        C4 level 2 + UML component
│   └── level-2/orders.md        one page per component
├── 06-runtime-view.md           sequence diagrams (Ch. 10)
├── 07-deployment-view.md        deployment diagrams (Ch. 12)
├── 08-crosscutting.md           security, persistence, events
├── 09-decisions/                ADR-001 ... ADR-007 (this slide)
├── 10-quality-scenarios.md      "extract Notification in 1 sprint"
├── 11-risks-technical-debt.md   cycle found by jdeps, 2026-09
└── 12-glossary.md               Order, Line, Payment token ...
```

- **Evolution**: a component diagram is a snapshot; ADRs explain *why* it looks the way it does and what was rejected.
- One ADR per decision, immutable once accepted; a new ADR *supersedes* an old one instead of editing it.
- Keep diagrams as **text** (PlantUML, Mermaid, Structurizr DSL) next to the code so they are diffed and reviewed like code.
- Generated diagrams (Spring Modulith `Documenter`, `jdeps` DOT) are the *as-is*; hand-drawn ones are the *to-be*. Compare them in every architecture review.

⚠ An architecture document without dates and decision records goes stale in weeks. A folder of ADRs stays useful for years.

Component × responsibility matrix: every row one owner, every column cohesive



- **Rows**: responsibilities from the use cases (Ch. 07) and sequence diagrams (Ch. 10). **Columns**: components.
- Rule 1: **exactly one tick per row**. Two ticks = duplicated logic or a hidden shared table.
- Rule 2: the ticks in a column must **belong together** (same actor, same data, same change rate). If you cannot name the column in two words, split it.
- Rule 3: no empty columns (a component with no responsibility is a wrapper) and no empty rows (a use case nobody implements).
- Validate by **walking a sequence diagram** across the components: every message must land on a provided interface of the target.

Question	Signal of a bad boundary
Who changes it?	two teams edit the same component weekly
What data does it own?	two components write one table
Can I replace it?	replacing means touching three other modules
Can I test it alone?	test needs half the system running

Component smells and the refactoring that fixes them

Smell	Symptom	Refactoring
God component	"Core" with 60% of the classes, everyone depends on it	split by responsibility rows; extract `shared` kernel
Anaemic component	only DTOs and pass-through calls	merge into its only client
Chatty connector	10 calls per use case across a boundary	coarser interface, or move the logic
Shared database	two components write the same table	one owner; others use API/events
Leaky interface	JPA entity or Stripe DTO in a provided interface	records / views in `api`
Cyclic dependency	A → B → A	invert with interface or event
Interface without consumer	lollipop that no socket uses	delete (YAGNI)

☑ Good boundary check: describe the component in one sentence without the word "and".

```
// BEFORE: OrderService owns stock AND orders (two ticks in one row)
@Transactional
public OrderId place(PlaceOrderCommand cmd) {
    for (var line : cmd.lines()) {
        // SQL against the catalogue's table from inside Orders
        jdbc.sql("UPDATE product SET stock = stock - ? WHERE id = ?")
            .params(line.qty(), line.productId().value()).update();
    }
    /* ... */
}

// AFTER: stock stays in Catalogue; Orders asks through its port
package edu.itc.shop.catalog.api;
public interface StockReservation {
    ReservationId reserve(List<Line> lines) throws OutOfStock;
    void release(ReservationId id);
}

// Orders: one required interface, one responsibility
@Transactional
public OrderId place(PlaceOrderCommand cmd) {
    var reservation = stock.reserve(cmd.lines());    // Catalogue
    var order = Order.from(cmd, reservation);
    var result = payment.charge(order.id(), order.total(),
                                cmd.token());          // Payment
    if (result instanceof PaymentGateway.Declined d) {
        stock.release(reservation);
        throw new PaymentDeclined(d.reason());
    }
    repo.save(order.paid());
    events.publishEvent(OrderPlaced.of(order));         // Notification
    return order.id();
}
```

After the change the responsibility matrix has one tick per row, the diagram gains one lollipop (`StockReservation` on Catalogue) and one socket (on Orders), and `jpa_stays_in_adapters` passes again.

Best practices and common mistakes

1 Name the interfaces, not just the boxes

A component diagram without lollipops and sockets is a box diagram. `OrderApi`, `PaymentGateway`, `OrderEvents` are the real architecture.

2 Let the callee's owner define required interfaces

Ports (`spi`) belong to the module that needs them, written in its language. Adapters translate; the domain never imports vendor types.

3 Make the diagram executable

Maven modules + `module-info.java` + ArchUnit / Modulith `verify()` + `jdeps` in CI. If it is not enforced, it is a wish.

4 Keep the graph acyclic and pointing at stability

Stable modules hold interfaces and value types. Break cycles with events or interface inversion, never with a "common" module that grows without limit.

5 One owner per table, per responsibility

Fill in the responsibility matrix before drawing. Two ticks in a row is the most common cause of a future "distributed monolith".

6 Document decisions, not only shapes

C4 for the overview, UML components for precision, an ADR for every boundary you argued about. Store all three as text next to the code.

✕ Mistake: class diagram in disguise

Forty boxes named after classes. A component diagram has 5-12 units; if it has more, you are one zoom level too deep.

✕ Mistake: dashed arrows everywhere

"Uses" arrows between business components hide what is used. Replace each with a named provided/required pair, or a published event.

✕ Mistake: microservices first

Extracting a service before the in-process boundary is clean turns method calls into network calls and cycles into outages. Modular monolith first.

Chapter quiz 10 questions

1. In UML 2.5, how is a provided interface drawn on a component?

- A dashed arrow with an open head
- A lollipop: small circle on a stub line (○—)
- A half-circle socket (—()
- A small square on the border

2. An assembly connector joins…

- two ports of the same component
- an artifact to the component it manifests
- a required interface (socket) to a compatible provided interface (ball)
- a subsystem to an external system

3. Which JPMS declaration corresponds to a required interface that another module will implement?

- exports
- opens … to
- uses
- requires transitive

4. In a hexagonal architecture, in which direction do all dependencies point?

- Downwards, from presentation to infrastructure
- Outwards, from the domain to the adapters
- Into the domain: adapters depend on ports the domain declares
- Along the layers, left to right

5. What is the main problem with a dashed «use» arrow from Notification straight to Order Management?

- It is not valid UML
- It does not say what is used, so internals are exposed and the boundary cannot be enforced
- Dashed arrows may only be used for external systems
- It implies an asynchronous call

6. Order Management depends on Notification and Notification depends on Order Management. The recommended fix is…

- merge both into one component
- let Order Management publish an OrderEvents interface / event that Notification requires
- add a third "common" module that imports both
- mark the dependency as «transitive»

7. Which ArchUnit rule fails when there is a dependency cycle between component packages?

- slices().matching("edu.itc.shop.(*)..").should().beFreeOfCycles()
- noClasses().should().dependOnClassesThat().resideInAPackage("..web..")
- classes().should().bePublic()
- layeredArchitecture().consideringAllDependencies()

8. What does "Used undeclared dependencies" from mvn dependency:analyze tell you?

- A declared dependency is never used and should be removed
- The code uses a library that arrives only transitively, so the diagram is missing an edge
- A module has a cycle
- A test-scoped dependency leaked into main code

9. A C4 container diagram differs from a UML component diagram mainly because it…

- shows people and deployable containers with technology labels instead of interfaces and connectors
- is the same thing under another name
- only shows classes
- cannot show external systems

10. In a component × responsibility matrix, a row with two ticks means…

- good redundancy for availability
- the responsibility has two owners: duplicated logic or a shared table that must be resolved
- the component is a library
- the row belongs to an external system

WRAP-UP

Summary

A component is a replaceable unit known only through its **provided** (○—) and **required** (—() interfaces, attached to ports.

Assembly connectors plug a socket onto a lollipop; delegation connectors forward a port to an internal part.

Prefer named interface connectors over dashed «use» arrows between business components; keep infrastructure and external systems outside the subsystem.

Layered and hexagonal are two internal structures; in hexagonal, ports belong to the domain and every arrow points inwards.

Stereotypes («service», «library», «database», «external», «queue») say how a unit is deployed and replaced; the same diagram serves monolith and microservices.

One component = one Maven module = one JPMS module: `exports`/`requires`/`provides … with` mirror the diagram; Spring Modulith `verify()` checks it.

Dependencies form a DAG pointing at stable, abstract modules; break cycles by inverting with an interface or event (ArchUnit `beFreeOfCycles`).

Verify the built artifact with `jdeps`, `mvn dependency:analyze` and Testcontainers boundary tests; the deployment view (Ch. 12) must agree with the component view.

Document with C4 + UML + ADRs stored as text; validate responsibilities with a component × responsibility matrix (one tick per row).

Open Lab-11: 5 tasks + 1 challenge

Next chapter: 12 · Deployment Diagrams